UNITÀ 2 · TEST DI WEB SERVICE REST CON CURL E POSTMAN

Collaudatore di API per un giorno

Un'azienda ti affida il collaudo di BiblioAPI prima del rilascio. Il tuo lavoro: trovare in modo sistematico ciò che funziona, ciò che no, e lasciare una collezione che chiunque possa rieseguire.

Cosa consegni: un file **collaudo-cognome.zip** con la collezione esportata (Bruno: la cartella; Postman: il file JSON v2.1) e il rapporto **collaudo-cognome.pdf** su Novaula, nella scheda Compiti. **Scadenza:** quella indicata su Novaula.

Che cosa devi fare

1. Parti dalla collezione fatta in classe. Usa le variabili `{{base}}` e `{{id}}`: nessun id scritto a mano.
2. Porta la collezione ad almeno **12 richieste**, ciascuna con almeno un test sul codice di stato e, dove c'è un corpo, un test sul contenuto (un campo, il tipo di un valore, la lunghezza di un array).
3. Almeno **6 casi negativi**: id inesistente, id non numerico, campi mancanti, JSON non valido, metodo non previsto, risorsa inesistente, titolo vuoto, anno negativo…
4. Esegui tutta la collezione con il Runner, a server appena riavviato, e fai uno screenshot del risultato.
5. Nel PDF scrivi la **tabella dei casi di test** (richiesta, risultato atteso, risultato ottenuto, esito) e la sezione **Difetti trovati**: per ogni test fallito, che cosa fa il server, che cosa dovrebbe fare, come lo correggeresti.

Esempio di riga del rapporto

N.	Richiesta	Atteso	Ottenuto	Esito
9	`POST {{base}}/libri` con `{"titolo":"","autore":"X","anno":2000}`	400	201	Difetto: il server accetta un titolo vuoto

Attenzione: collauda solo il tuo server su `localhost` o le API pubbliche fatte per esercitarsi, come JSONPlaceholder. Mandare richieste malformate in massa a servizi altrui senza permesso non è un test: può essere un reato.

> *"Il test dei programmi può servire a mostrare la presenza di bug, ma mai a dimostrarne l'assenza!"* · **Edsger W. Dijkstra**, *Notes on Structured Programming, 1970 (traduzione nostra)*

Come viene valutato

Criterio	Punti
Almeno 12 richieste con variabili, rieseguibili senza modifiche	2
Test significativi su codice e contenuto	2
Almeno 6 casi negativi ben scelti	2
Rapporto: tabella completa e screenshot del Runner	2
Difetti trovati descritti con atteso, ottenuto e correzione proposta	2

Non trovi difetti? Allora cerca meglio: il BiblioServer scritto in classe, per esempio, accetta un libro dell'anno 3000 e perfino un JSON non valido, purché contenga i tre campi.